

Applying Constraint Programming to column mapping

- [Introduction](#)
- [1. Column mapping as a CP problem](#)
 - [1.1. Pivot table](#)
 - [1.2. Mapping model](#)
 - [1.3. Fuzzy mapping](#)
 - [1.4. Implementation of the model](#)
 - [1.5. Running program and getting results](#)
- [2. Managing interactions with CP framework](#)
 - [2.1. CPSMapping processor project](#)
 - [2.1.1. Project structure](#)
 - [2.1.2. CPSMapping processor implementation](#)
 - [2.2. Test bench](#)
 - [2.2.1. Pivot table output](#)
 - [2.2.2. Generating mask hash map](#)
 - [2.2.3. Running CPSMapping processor with mask map](#)
 - [2.2.4. Back to the Test bench to continue pruning](#)
- [References](#)
- [Appendix 1. Introduction Program code](#)
- [Appendix 2. Results of execution CPSMapping.java program](#)
- [Appendix 3. Source code of CPSMapping processor](#)
- [Appendix X. Initial notes](#)

Introduction

Introduction and the full program code is in the [Appendix 1](#).

1. Column mapping as a CP problem

1.1. Pivot table

Pivot table is a tabular presentation of the relations between columns from different tables. There are two ways to represent these relations:

1. Using **cardinality** of intersections of matched columns;
2. Using the values of a **matching coefficient** of related columns.

Here is an example of the match coefficients pivot table:

	1	2	3	4	5	6	7	8	9	10	11	12
1			66					14				
2		99						45		19		
3			40						17			
4				32							12	
5		72	100									
6		57		24				40				
7				22		56						
8					100							
9												72
10	72								34			

Disclaimer. The data was taken from a real match matrix but it was modified and serves only for the illustration purposes.

1.2. Mapping model

Legend:

1. **Rows** of the pivot table represent columns from the **source table**;
2. **Columns** are representing columns from **big table**.

Columns from source table are sub-set of the all columns and they were extracted by applying a grouping algorithm. It means that sub_set has no matching elements pairwise, for each two columns (i, j) from the pivot table we have:

$$\text{match}(i, j) \leq \text{threshold, threshold is closed to 0.}$$

This leads us to the first assumption (**constraint 1**):

Source column cannot be mapped into more than one big table column.

Also it is hard to imagine that ETL processing may be design a such way that data from two source columns will go into one big table column. So, we can set another **constraint 2**:

Big table column can be sourced only from one source column.

In order to satisfy **constraint 1** the following condition should be met:

$$c(i, 1) * d(i, 1) + c(i, 2) * d(i, 3) + \dots + c(i, n) * d(i, n) \max.$$

where:

$c(i, j)$ is a coefficient (unknown variable) that we are looking for, and a such that it take values from {0, 1};

$d(i, j)$ is a known data element from from above matrix.

Effectively it means that the sum (by rows) of all $c(i, j)$ coefficients should be equal 1.

The **constraint 2** is a transposition of constraint 1.

$$c(1, j) * d(1, j) + c(2, j) * d(3, j) + \dots + c(m, j) * d(m, j) \max.$$

where:

$c(i, j)$ is a coefficient (unknown variable) that we are looking for, and a such that it take values from {0, 1};

$d(i, j)$ is a known data element from from above matrix.

Effectively it means that the sum (by columns) of all $c(i, j)$ coefficients should be equal 1.

1.3. Fuzzy mapping

In the ideal world constraints defined in the previous section are more the enough. We are living in the world that is far from ideal, so, the one-to-one mapping declared before will not hold the ground in too many cases.

The fuzzy mapping is the way to address “fuzziness” of the real world data.

We can implement fuzzy mapping as a CP model that can map source columns to more than one target columns. All we need to do is to allow sum (by columns or by rows) of all $c(i, j)$ coefficients **be equal or greater than 1**. In this case CP solver will map each source column to more than one target column.

1.4. Implementation of the model

Here is a code that implements the model and constraints declared above:

1. Set range for all coefficients:

```

// Build coeff matrix and sum vectors for matrix rows and columns.
//=====
IntVar[][] coeff = new IntVar[ot_idx][bt_idx];
for (int j = 0; j < ot_idx; j++) {
    for (int i = 0; i < bt_idx; i++) {
        String cell = j + "_" + i;
        coeff[j][i] = model.newIntVar(0, 1, cell);
    }
}
// Sum vector should compensate
// possible mismatch between ot_column cardinality and cardinality of
// bt_column and ot_column intersection.
//=====
IntVar[] sum_row = new IntVar[ot_idx];
for (int j = 0; j < ot_idx; j++) {
    int ub = ot_col_set.get(ot_col_idx.get(j)).getCard();
    sum_row[j] = model.newIntVar(0, ub, "sum_row_" + j);
}
IntVar[] sum_col = new IntVar[bt_idx];
for (int j = 0; j < bt_idx; j++) {
    int ub = bt_col_set.get(bt_col_idx.get(j)).getCard();
    sum_col[j] = model.newIntVar(0, ub, "sum_col_" + j);
}

```

2. Build other constraints:

```

// Building constraints
//=====
// 1. Sum of coefficints in each row should be equal 1 ('ot' column can
be
// mapped only to one 'bt' column). In fuzzy mapping this number can
be greater than 1;
// 2. Scalar product of data row on 'bt_idx_vars'
// should be equal 'ot' column cardinality minus delta.
// Cardinality of intersection cannot be greater than cardinality of
any set
// in intersection.
//=====
for (int i = 0; i < ot_idx; i++) {
    // 0. Create IntVar array by row
    IntVar[] bt_idx_vars = new IntVar[bt_idx];
    System.arraycopy(coeff[i], 0, bt_idx_vars, 0, bt_idx);
    // 1. row_ub equal 1, in standard case; col_ub > 1 in fuzzy mapping
    model.addLessOrEqual(LinearExpr.sum(bt_idx_vars), row_ub);
    model.addGreaterOrEqual(LinearExpr.sum(bt_idx_vars), 0);
    // 2.
    model.maximize(LinearExpr.scalProd(bt_idx_vars, data[i]));
    model.addEquality(
        // scalarProduct should be equal to delta
        LinearExpr.scalProd(bt_idx_vars, data[i]),
        // delta scope is defined as [0, card_of_ot_column].
        sum_row[i]
    );
}
long[][] t_data = transposeMatrix(data);
for (int i = 0; i < bt_idx; i++) {
    // 0. Create IntVar array by col
    IntVar[] ot_idx_vars = new IntVar[ot_idx];
    for (int j = 0; j < ot_idx; j++) {
        ot_idx_vars[j] = coeff[j][i];
    }
    // 1. col_ub equal 1, in standard case; col_ub > 1 in fuzzy mapping
    model.addLessOrEqual(LinearExpr.sum(ot_idx_vars), col_ub);
    model.addGreaterOrEqual(LinearExpr.sum(ot_idx_vars), 0);
    // 2.
    model.maximize(LinearExpr.scalProd(ot_idx_vars, t_data[i]));
    model.addEquality(
        // scalarProduct should be equal to delta
        LinearExpr.scalProd(ot_idx_vars, t_data[i]),
        // delta scope is defined as [0, card_of_ot_column].
        sum_col[i]
    );
}

```

3. Create a solver and solve the problem:

```
// Create a solver and solve the model.
//=====
=====
CpSolver solver = new CpSolver();
VarArraySolutionPrinter cb = new VarArraySolutionPrinter(coeff);
solver.solveWithSolutionCallback(model, cb);
```

1.5. Running program and getting results

The results of running this program against real data are in the Appendix 2.

Here is sub-matrix from one of the solutions.

	1	2	3	4	5	6	7	8	9	10	11	12
1			1									
2		1										
3									1			
4				1								
5												
6								1				
7						1						
8					1							
9												1
10	1											

This is one of the choices that the algorithm made for us. This is not the final decision and we always can except it or reject this proposal.

The easiest way to declare our choice is to fix the value of a specific coefficient from the matrix **coeff** [][]:

- **set a selected c (i, j) to 0**, if we want to reject CP framework choice, or we are confident that specified relation is invalid (it doesn't matter, if algorithm marks this relation as 0 or 1; if we set it, this would be our choice that CP framework will respect in all following generated solutions);
- or **set a selected c (i, j) to 1**, if we made a decisions that this specific relation is a valid one.

We can do these settings at any time of an interactive process of finding the best mapping from source columns to the target columns.

Here is a modified code that performs initial matrix **coeff** [][] assignment. This code takes a user feedback into consideration:

```

// Build coeff matrix and delta vector.
// mask_map is a list for pre-sets for selected relations
// between source and target columns.
//=====
IntVar[][] coeff = new IntVar[ot_idx][bt_idx];
for (int j = 0; j < ot_idx; j++) {
    for (int i = 0; i < bt_idx; i++) {
        String cell = j + "_" + i;
        // Check if the value of this cell is in the user feedback
        //=====
        if (mask_map.get(cell) == null) {
            coeff[j][i] = model.newIntVar(0, 1, cell);
        } else {
            // Setting IntVar as a constant
            int v = (int) mask_map.get(cell);
            coeff[j][i] = model.newIntVar(v, v, cell);
        }
    }
}
}

```

Here is a function that generates **mask_map** from provided JSON file located at the `mask_path`:

```

private static ImmutableMap<String, String> createMaskMap(String
mask_path) {
    Properties props = new Properties();
    try (InputStream input = new FileInputStream(mask_path)) {
        // load a properties file
        props.load(input);
    } catch (IOException ex) {
        // Do nothing and return map from empty props
    }
    return Maps.fromProperties(props);
}

```

CP framework treats constant as a special case of a variable.

For example:

```
model.newIntVar(1, 1, "var");
```

makes variable with a name "var" equal 1 all the time, and

```
model.newIntVar(0, 0, "var");
```

makes this constant equal 0.

CP works with each variable from matrix **coeff** `[][]` independently, and if we set some matrix element in the row `i` as a **constant** equal 1, it will guaranty that all other elements in this row will be equal 0 in order to satisfy equality constraints:

```
model.addLessOrEqual(LinearExpr.sum(ot_idx_vars), 1);  
model.addGreaterOrEqual(LinearExpr.sum(ot_idx_vars), 0);
```

This constraints would be preserved in all generated solutions.

The same would be true for by-column constraints.

```
model.addLessOrEqual(LinearExpr.sum(bt_idx_vars), 1);  
model.addGreaterOrEqual(LinearExpr.sum(bt_idx_vars), 0);
```

2. Managing interactions with CP framework

The interaction process will follow the same scenario that we are thinking about right now. The only difference would be a way how we will handle a user feedback.

In case of CP framework the user feedback should will be presented as a JSON property file (as a representation of the mask_map), and it should be passed to the CP framework after each iteration.

The mask_map defines pruning of the CP proposed mapping between source and target columns; user should mark all suggestions that are wrong from his/her point of view.

This map will be delivered to the CP framework, so in the next run pruned relations would be respected, and they will not be propagated to next iterations.

We have to keep a history of all mask_map(s), and provide a way for the user to navigate through the list of saved mask_map(s).

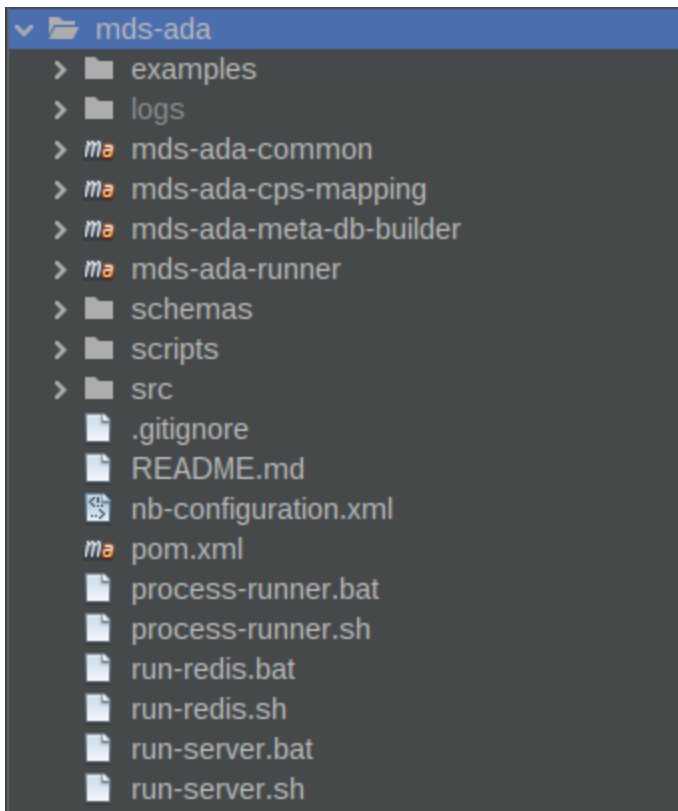
CP support was moved to the **mds-ada-cps-mapping** module of the **mds-ada** project.

The **halo-bkind-mapping-gui** project is also updated. This is the project that we are using to test and verify our data analysis strategies.

2.1. CPSMapping processor project

2.1.1. Project structure

In the latest restructuring of the mds-ada project, we moved all processors into separate modules:



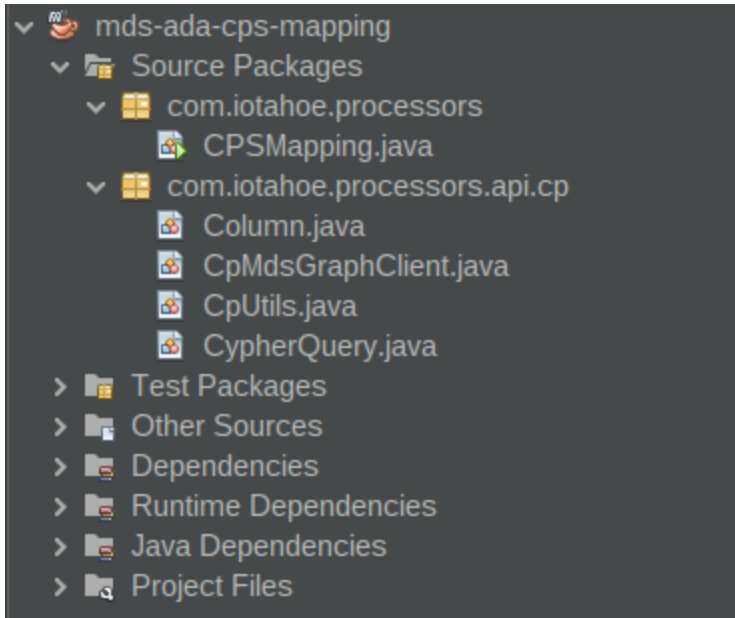
There are two groups of the projects:

1. Core:
 - a. mds-ada-common;
 - b. mds-ada-runner;
2. Processors:
 - a. mds-ada-meta-db-builder;
 - b. mds-ada-cps-mapping
 - c. other processors we are going to move in couple of weeks.

This architecture makes Processor's projects totally independent and they can be developed separately.

Mds-ada core modules isolate resources from processors. There is no direct calls from any processor to the resources like Redis, MongoDB, or any others.

Here is out-line of the **mds-ada-cps-mapping** project:



Here:

CPSMapping.java is the processor class;

CpMdsGraphClient.java and **CpUtils.java** are extensions of the classes (**MdsGraphClient** and **Utils**) from **mds-ada-common** project.

Column.java and **CypherQuery.java** are supporting classes.

The other processor's projects would look similar.

2.1.2. CPSMapping processor implementation

CPSMapping processor is taking the following set of parameters as an input:

```
{
  "schemaName": "CPSMapping",
  "name": "CPS Mapping",
  "description": "Builds pivot table (mapping) between columns of two
table, or datasets using OR-Tools Constraint Programming framework.",
  "org_table": "GRP02_SOURCE2",
  "big_table": "BIG_TABLE",
  "row_ub": 3,
  "col_ub": 3,
  "threshold": 20,
  "match": true,
  "graph_name": "MDS_GRAPH",
  "pivot_matrix": "pivot_matrix",
  "pivot_map": "pivot",
  "mask_hash": "mask",
  "lettuce_host": "redis://localhost",
  "redis_host": "localhost",
  "redis_port": 6379,
  "hello": "HelloWorld!"
}
```

The following is the resulting map that connect columns from an original table to the big table as a target. This map is represented as Redis hash:

```
127.0.0.1:6379> hgetall pivot:GRP02_SOURCE2:3:3:10
1) "df2cf8c37bdd864e3d07b4b975478827cfbefffd:
0449370eb51b53f5a312ce1c3a1f6d5746404a08"
2) "28"
3) "b20faf24826745cc586ce9b5ed6a38981a90d9d7:
590b0c4d006226e0785ce512d2f8ecd95f3a1d7d"
4) "13"
5) "cfae644897d7efe12b0aae145378ad2de9f23e4f:
9cc819c4dc251c400050333fc32d5264829df523"
6) "100"
7) "9f56ae83a775cdb475fe90ab2c7a38a0713276e6:
cefb3d64e07f3a5c8ba2cf24814fba41fb8a19cd"
8) "17"
9) "19917584c86298495541e5bb12dec1a45f931026:
f14b788797293fbfcb6a653c9b61d4a90361000f1"
10) "17"
11) "ba4925e64c67e12280add35cd21a998baba6088b:
81d7f4bca69981a0eeb6221b28dd04a961665e74"
12) "97"
. . . . .
109) "0d8d02076745ba36225ae0330de195c055d715c1:
59a5163c04b0df417185dd4b7dab4098c762a8cb"
110) "18"
111) "1b1fca56f5b60a71150d7de1254420886396a86c:
2560d3cb43d3bf987894fefee9230c45a65af1c6"
112) "16"
113) "84999bdb4e0d8eedf2c7adcbeffb95801656bb04:
f5e7e24d4c55b92c6fe914ff9ba7a6ecb52e6a07"
114) "13"
```

The **pivot hash map name** is a composition of the prefix pivot an **original table name** (GRP02_SOURCE2) , a **row upper bound** (3), a **column upper bound** (3), and **threshold** for minimal matching value (10).

Row (and column) upper bound is a max number of non zero elements in each row (column) of the mapping matrix.

In the resulting map each row from the matrix is a group of three entries. Below is an example of the first row from the matrix:

Map key	Map value
"df2cf8c37bdd864e3d07b4b975478827cfbefffd: 0449370eb51b53f5a312ce1c3a1f6d5746404a08"	"28"
"b20faf24826745cc586ce9b5ed6a38981a90d9d7: 590b0c4d006226e0785ce512d2f8ecd95f3a1d7d"	"13"
"cfae644897d7efe12b0aae145378ad2de9f23e4f: 9cc819c4dc251c400050333fc32d5264829df523"	"100"

As we mentioned before CP framework may produce more than one solution, the **map** is a merge of all generated solutions.

The full text of **CPSMapping.java** file from the “mds-ada-cps-mapping” project is in **Appendix 3**.

2.2. Test bench

2.2.1. Pivot table output

Below is a screenshot of a pivot table with the data loaded CPSMapping resulting map. Resulting map, as you remember, provides only column's IDs, the rest of the data in the pivot table were extracted from the MDS_GRAPH.

Selected Pivot Map: pivot:GRP02_SOURCE0:3:3:20

Dictionary	Left ID	Left_Match	M_Match	Left Name	Pivot	Right Name	Right ID	Right_Match	Mask
	ca4f99f2e0d22c5f6d4949f32187f52e9afa4384	90	8	validated_tax_amount		entered_dr	c9c275ae31d9871c64b2fe6c660693bd0a819fe5	8	☐
	a73df4c1322eab3c5573e406a65874e651ce9587	91	2	approved_amount		entered_cr	9f9a5fceddf61de5c9a0afa5ae88bbff582770f6	2	☒
	13c9dd9c91c827c0c4ccd56acd1bdcc7ea5720d1	96	1	base_amount		accounted_dr	81d7f4bca69981a0eeb6221b28dd04a961665e74	1	☒
	7a103c6689d813b3cb60b54be2c5e58b0dfc80e6	89	85	creation_date		creation_date	82dd99cd1b387d0385f6ad3cd364d4bb0a9d0c1c	95	☐
	859995888a9b3ab1f49ed46129c034bc937494a0	98	2	discount_amount_taken		entered_dr	c9c275ae31d9871c64b2fe6c660693bd0a819fe5	2	☒
	4fa64e5b8f71e86080f6c158cb4c8fd23347957	93	76	last_update_date		creation_date	82dd99cd1b387d0385f6ad3cd364d4bb0a9d0c1c	80	☐
	757b1c58370c3679be92235147b263f680d6383e	75	2	vendor_site_id		reference_4	2560d3cb43d3bf987894fefee9230c45a65af1c6	2	☒
	7a103c6689d813b3cb60b54be2c5e58b0dfc80e6	86	83	creation_date		last_update_date	f2d5d080207251fe6f753e9469d330d27270b06	96	☐
	4fa64e5b8f71e86080f6c158cb4c8fd23347957	95	48	last_update_date		effective_date	9cc819c4dc251c400050333fc32d5264829df523	49	☐
	a73df4c1322eab3c5573e406a65874e651ce9587	35	23	approved_amount		attribute6	3bd1493c58b86666633ada070928e24e2e7af8b0	41	☐
	4fa64e5b8f71e86080f6c158cb4c8fd23347957	92	77	last_update_date		last_update_date	f2d5d080207251fe6f753e9469d330d27270b06	82	☐
	a73df4c1322eab3c5573e406a65874e651ce9587	93	2	approved_amount		accounted_cr	0888d848048d6d98ac8c628d6fca9ea0298bb724	2	☒
	c93b130182eb61d3ea5a421258611a48d262e9	76	4	set_of_books_id		reference_8	213c5efbcbbd95d6cba6cfeff7bc7e98893a638f	4	☐
	757b1c58370c3679be92235147b263f680d6383e	100	4	vendor_site_id		subledger_doc_sequence_value	f14b788797293fbcba653c9b61d4a9036100f1	4	☐
	9479e40c0c3625799698e38db027110851f160c	41	38	created_by		last_updated_by	6696b3d96a9af239d0f2c2a386736665d90a32b6	85	☐
	3b9ec18e041af62d1fe858481996a8f6caba4d08	93	22	external_bank_account_id		reference_8	213c5efbcbbd95d6cba6cfeff7bc7e98893a638f	22	☐

Sample DataColumn GroupsPivot TableCP Pivot TableClear HistorySubmit GroupsSubmit PivotSubmit Cp Pivot

We got this table by pushing "CP Pivot Table" button. On the right side of the table are check-boxes that we can use to mask pairs that are not valid (false positives).

2.2.2. Generating mask hash map

After pushing "Submit Cp Pivot" we will generate a mask map as a hash in the Redis:

```
127.0.0.1:6379> hgetall mask:GRP02_SOURCE0:3:3:20
1) "a999c9095d87dae05abd455e8989406219513d6f:
63d287acd405b26c6deff7b815740a4e5e7ad923"
2) "0"
3) "3b9ec18e041af62d1fe858481996a8f6caba4d08:
f225438efaa199d86702799b830975353303c334"
4) "0"
5) "a73df4c1322eab3c5573e406a65874e651ce9587:
9f9a5fceddf61de5c9a0afa5ae88bbff582770f6"
6) "0"
7) "13c9dd9c91c827c0c4ccd56acd1bdcc7ea5720d1:
81d7f4bca69981a0eeb6221b28dd04a961665e74"
8) "0"
9) "859995888a9b3ab1f49ed46129c034bc937494a0:
c9c275ae31d9871c64b2fe6c660693bd0a819fe5"
10) "0"
11) "757b1c58370c3679be92235147b263f680d6383e:
2560d3cb43d3bf987894fefee9230c45a65af1c6"
12) "0"
13) "a73df4c1322eab3c5573e406a65874e651ce9587:
0888d848048d6d98ac8c628d6fca9ea0298bb724"
14) "0"
```

Mask hash map has a composite name and a structure similar to the pivot map hash, except the prefix is equal mask , and the value is equal 0. This indicates that this particular pair should be excluded from the next generated solutions produced by CPSMapping processor.

Map key	Map value
"a999c9095d87dae05abd455e8989406219513d6f: 63d287acd405b26c6deff7b815740a4e5e7ad923"	"0"
"3b9ec18e041af62d1fe858481996a8f6caba4d08: f225438efaa199d86702799b830975353303c334"	"0"
"a73df4c1322eab3c5573e406a65874e651ce9587: 9f9a5fceddf61de5c9a0afa5ae88bbff582770f6"	"0"
" :"	"0"

2.2.3. Running CPSMapping processor with mask map

Now, we can run CPSMapping processor again and get an updated result.

This is a final output after this run:

```
2021-05-16 17:40:14 INFO CPSMapping:169: MASK_MAP: {"50_10":0,"33_3":
0,"58_27":0,"10_3":0,"2_3":0,"31_1":0,
"31_11":0,"3_4":0,"29_6":0,"29_5":0,"3_7":0,"9_1":0,"4_7":0,"8_4":0,"
7_36":0,"7_25":0,"9_25":0,"7_37":0,
"58_23":0,"58_24":0,"25_16":0,"9_11":0}
2021-05-16 17:40:14 INFO CPSMapping:200: coeff_by_row count: 195
2021-05-16 17:40:14 INFO CPSMapping:201: coeff_by_col count: 42
2021-05-16 17:40:14 INFO CPSMapping:252: Statistics
2021-05-16 17:40:14 INFO CPSMapping:253: - conflicts : 0
2021-05-16 17:40:14 INFO CPSMapping:254: - branches : 1411
2021-05-16 17:40:14 INFO CPSMapping:255: - wall time : 0.201902239 s
2021-05-16 17:40:14 INFO CPSMapping:256: - obj value : 13110.0
2021-05-16 17:40:14 INFO CPSMapping:257: - solutions : 2
2021-05-16 17:40:14 INFO CPSMapping:270:

runOnce: finished; seconds=6.6327147
```

At the top of this output is a mask map that we passed to the CPSMapping processor. This mask map is converted from the previous one. In this map column IDs are replaced with the matrix coordinates - <row-index_column-index>.

```
MASK_MAP: {"50_10":0,"33_3":0,"58_27":0,"10_3":0,"2_3":0,"31_1":0,
"31_11":0,"3_4":0,"29_6":0,"29_5":0,"3_7":0,"9_1":0,"4_7":0,"8_4":0,"
7_36":0,"7_25":0,"9_25":0,"7_37":0,
"58_23":0,"58_24":0,"25_16":0,"9_11":0}
```

2.2.4. Back to the Test bench to continue pruning

Lets refresh Pivot table in Test bench by clicking on "CP Pivot Table" button:

Selected Pivot Map: pivot:GRP02_SOURCE0:3:3:20

Dictionary	Left ID	Left_Match	M_Match	Left Name	Pivot	Right Name	Right ID	Right_Match	Mask
6d2f48d7cfaddf18922247186e3f04bdf7a	96	72		goods_received_date	effective_date	9cc819c4dc251c400050333fc32d5264829df523	74		☐
69fc3bebc50250e26a796279d09881f442090a0f9	76	2		remit_to_supplier_site_id	reference_4	2560d3cb43d3b987894fefe9230c45a65af1c6	2		☐
f5a836bea236aaca0b7e42b4526faeeead3964f8	83	78		invoice_received_date	creation_date	82dd99cd1b387d0385f6ad3cd364d4bb0a9d0c1c	92		☐
13c9dd9c91c827c0c4cc5f6acd1bdcc7ea5720d1	92	1		base_amount	accounted_cr	0888d848048d6d98ac8c62896fc9ea0298bb724	1		☐
f5a836bea236aaca0b7e42b4526faeeead3964f8	98	65		invoice_received_date	effective_date	9cc819c4dc251c400050333fc32d5264829df523	65		☐
f585f9a351e14b039af7440b5a9c98efc2af9799	50	1		doc_sequence_id	reference_8	213c5ef8cbdd95d6cbaa6ceff7bc7e98893a638f	1		☐
ee3bd7e14300ac355cf1105e4113ea9317b9e5cf	100	60		doc_sequence_value	subledger_doc_sequence_value	f14b788797293fbcba653c9b61d4a90361000f1	60		☐
ee3bd7e14300ac355cf1105e4113ea9317b9e5cf	100	44		doc_sequence_value	je_line_num	343045a724baac87f232fb9f700b681c676a5da	44		☐
281e2d2269ff40b3f5cfe79a4393d9058183e5a	42	28		invoice_id	attribute2	cefb3d64e0773a5c8ba2cf24814fba41fb8a19cd	45		☐
a999c9095d87dae05abd455e8989406219513b6f	99	4		amt_due_employee	reference_4	2560d3cb43d3b987894fefe9230c45a65af1c6	4		☐
859995888a9b3ab1f49ed46129c034bc937494a0	99	2		discount_amount_taken	accounted_dr	81d7f8bca69981a0eeb6221b28d904a961665e74	2		☐
13c9dd9c91c827c0c4cc5f6acd1bdcc7ea5720d1	93	1		base_amount	entered_dr	c9c275ae31d9871c64b2fe6c660693bd0a819fe5	1		☐
a999c9095d87dae05abd455e8989406219513b6f	80	2		amt_due_employee	reference_9	f5e7e24d4c55b92c6fe914f9ba7a6eb52e6a07	2		☐
86bcd211167d2564a1c221e288dd3115249f937	89	10		total_tax_amount	entered_dr	c9c275ae31d9871c64b2fe6c660693bd0a819fe5	10		☐
7a103c66898813b3cb60e54be2c5e8b0d4c80e6	89	85		creation_date	creation_date	82dd99cd1b387d0385f6ad3cd364d4bb0a9d0c1c	95		☐

All pairs from mask map now removed from the pivot map.

We can go through this iterative process until we get result that will satisfy us, but this result should be in the pivot map to be discovered. Otherwise, we must accept the hard truth that there is no solution.

References

1. <https://developers.google.com/optimization/introduction/java>
2. <https://developers.google.com/optimization/install>
3. <https://choco-solver.readthedocs.io/en/latest/index.html>
4. https://github.com/iotahoe/kiot-java-amproc/tree/r_search2/src/main/java/com/iotahoe/processors/main/cp
5. <https://github.com/iotahoe/halo-bkind-mapping-gui/tree/master/src/main/java/com/iotahoe/mdsada/cp> (reference UI test bench)
6. <https://developers.google.com/optimization/lp/glop>
7. https://developers.google.com/optimization/mip/integer_opt
8. https://developers.google.com/optimization/cp/cp_solver

Appendix 1. Introduction Program code

Constraint programming (**CP**) is a generalized solution framework for many applied math problems. The concept is pretty simple:

1. We have some **data** and we know that different parts of these data is not independent. It means that changes in one part can affect other data;
2. We have some idea what kind of dependencies define relations between different parts of data (like linear, combinatoric, etc). We can treat these dependencies as **unknown variables**;
3. The value range of these variables is limited and we can make some estimations. This will allow us to define set of **constraints** that will limit area where we are going to search for **solutions**.

So, in the essence we have:

CP (data, variables, constraints) solutions.

The basic steps for setting up and solving a CP problems are the same in any programming language:

- Import the required libraries.
- Declare the solver.
- Create the variables.
- Define the constraints.
- Define the objective function.
- Invoke the solver and display the results.

Here is an example of a very basic Java program that implements a CP solution for linear optimization task (**taken directly from [1]**):

0		x	1
0		y	2
		x + y	2

1. Import the libraries:

```
import com.google.ortools.Loader;
import com.google.ortools.linearsolver.MPConstraint;
import com.google.ortools.linearsolver.MPObjective;
import com.google.ortools.linearsolver.MPSolver;
import com.google.ortools.linearsolver.MPVariable;
```

1. Declare the Solver:

```
// Create the linear solver with the GLOP backend.
MPSolver solver = MPSolver.createSolver("GLOP");
```

1. Create the variables:

```
// Create the variables x and y.
MPVariable x = solver.makeNumVar(0.0, 1.0, "x");
MPVariable y = solver.makeNumVar(0.0, 2.0, "y");

System.out.println("Number of variables = " + solver.numVariables());
```

1. Defining the constraints:

The first two constraints, $0 \leq x \leq 1$ and $0 \leq y \leq 2$, are already set by the definitions of the variables. The following code defines the constraint $x + y \leq 2$:

```
// Create a linear constraint,  $0 \leq x + y \leq 2$ .
MPConstraint ct = solver.makeConstraint(0.0, 2.0, "ct");
ct.setCoefficient(x, 1);
ct.setCoefficient(y, 1);
System.out.println("Number of constraints = " + solver.
    numConstraints());
```

The method `setCoefficient` sets the coefficients of **x** and **y** in the expression for the constraint.

1. Define the objective function:

```
// Create the objective function,  $3 * x + y$ .
MPObjective objective = solver.objective();
objective.setCoefficient(x, 3);
objective.setCoefficient(y, 1);
objective.setMaximization();
```

1. Invoke the solver and show results:

```

solver.solve();
System.out.println("Solution:");
System.out.println("Objective value = " + objective.value());
System.out.println("x = " + x.solutionValue());
System.out.println("y = " + y.solutionValue());

```

```

import com.google.ortools.Loader;
import com.google.ortools.linearsolver.MPConstraint;
import com.google.ortools.linearsolver.MPObjective;
import com.google.ortools.linearsolver.MPSolver;
import com.google.ortools.linearsolver.MPVariable;
/** Minimal Linear Programming example to showcase calling the solver.
 */
public final class BasicExample {
    public static void main(String[] args) {
        Loader.loadNativeLibraries();
        // Create the linear solver with the GLOP backend.
        MPSolver solver = MPSolver.createSolver("GLOP");
        // Create the variables x and y.
        MPVariable x = solver.makeNumVar(0.0, 1.0, "x");
        MPVariable y = solver.makeNumVar(0.0, 2.0, "y");
        System.out.println("Number of variables = " + solver.
numVariables());
        // Create a linear constraint, 0 <= x + y <= 2.
        MPConstraint ct = solver.makeConstraint(0.0, 2.0, "ct");
        ct.setCoefficient(x, 1);
        ct.setCoefficient(y, 1);
        System.out.println("Number of constraints = " + solver.
numConstraints());
        // Create the objective function, 3 * x + y.
        MPObjective objective = solver.objective();
        objective.setCoefficient(x, 3);
        objective.setCoefficient(y, 1);
        objective.setMaximization();
        solver.solve();
        System.out.println("Solution:");
        System.out.println("Objective value = " + objective.value());
        System.out.println("x = " + x.solutionValue());
        System.out.println("y = " + y.solutionValue());
    }
    private BasicExample() {}
}

```

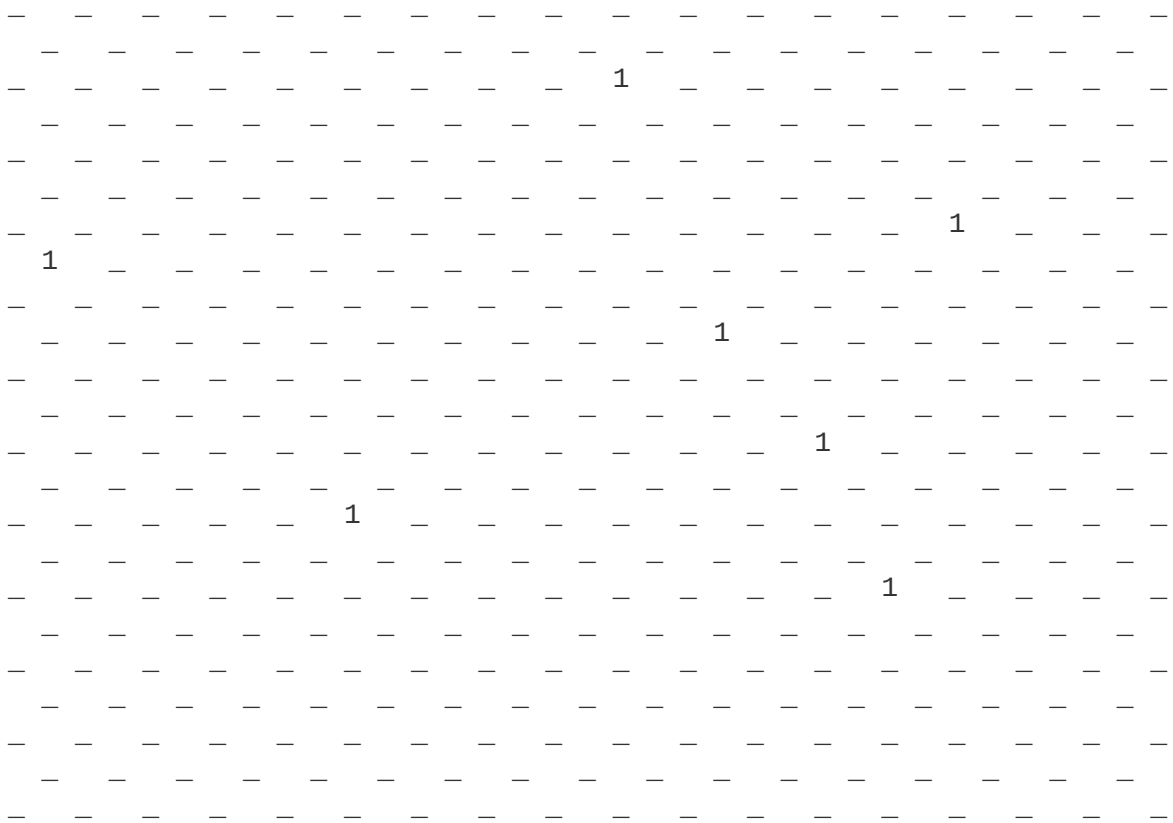
```
// Column IDs from BIG_TABLE
//=====
=====
{
0=99b1c64c0b314e42ed899a8b267434b5b76553c7,
1=f14b788797293fbfcba653c9b61d4a90361000f1,
2=5e9940dd08ccedd80ef06f06450e493fa6bbd038,
3=f225438efaa199d86702799b830975353303c334,
4=0888d848048d6d98ac8c628d6fca9ea0298bb724,
5=81d7f4bca69981a0eeb6221b28dd04a961665e74,
6=9f9a5fceddf61de5c9a0afa5ae88bbff582770f6,
7=c9c275ae31d9871c64b2fe6c660693bd0a819fe5,
8=9121741276b9535408af330f042beee8299e72b5,
9=cca24377d9a3bbf8e3c89235855698f649f37e4f,
10=343045a724baac87f232fbe9f700b681c676a5da,
11=0c223f2aa8007db68aa91921e981302db78d113e,
12=6696b3d96a9af239d0f2c2a386736665dd0a32b6,
13=95b1bfc84ee0252d5737fcbac2adac205272a149,
14=ab1d2c86440190b2d76baa9a294a2ffef1f703d6,
15=c0da906ffa025c1a85f5ae8966bde69117b2d2a9,
16=f5e7e24d4c55b92c6fe914ff9ba7a6ecb52e6a07,
17=213c5efbcbdd95d6cbaa6ceff7bc7e98893a638f,
18=3b9c1dc63c247964e5c4bbdf2d3703801bbaafb,
19=dfb049f0f1e9617bf99fb4e127ca66cf0f18cd68,
20=5f18c73c21c2fa0711966614d82eefcdfea5776e,
21=8a3d2f165b9fb8aea4d077afd11c79c9a6eccf5b,
22=255704a9ec7c49b1ede3703aa2944069eaf05c9d,
23=0f24c293e170644aa17a19cb783f863ec2c523fa,
24=3bd1493c58b86666633ada070928e24e2e7af8b0,
25=cefb3d64e07f3a5c8ba2cf24814fba41fb8a19cd,
26=a0568ddac2f098ef56c83e55ffde0b8e578228b3,
27=75f66fd3b641283cddc3bcad1ceef7d9c645df6c,
28=2560d3cb43d3bf987894fefee9230c45a65af1c6,
29=590b0c4d006226e0785ce512d2f8ecd95f3a1d7d,
30=0449370eb51b53f5a312ce1c3a1f6d5746404a08,
31=63d287acd405b26c6def7b815740a4e5e7ad923,
32=59a5163c04b0df417185dd4b7dab4098c762a8cb,
33=6bd679ad87136393c21ebfffb8c25a3eea6341ca,
34=3b8afb9005bb0b434e498fe7726a78939552a145
}
// Columns from GRP02_SOURCE0 table
//=====
=====
{
0=ca4f99f2e8d22c5f6d4949f32187f52e9afa4384,
1=a999c9095d87dae05abd455e8989406219513d6f,
2=ff481ec5c2d88ab48d63747db4a4b1439809ac67,
3=3b861eb59fc8723ae73bb942e8d1d212e286f787,
4=0ed4eaad45b9a2274f446c75fa3c3a743d290a16,
```



```

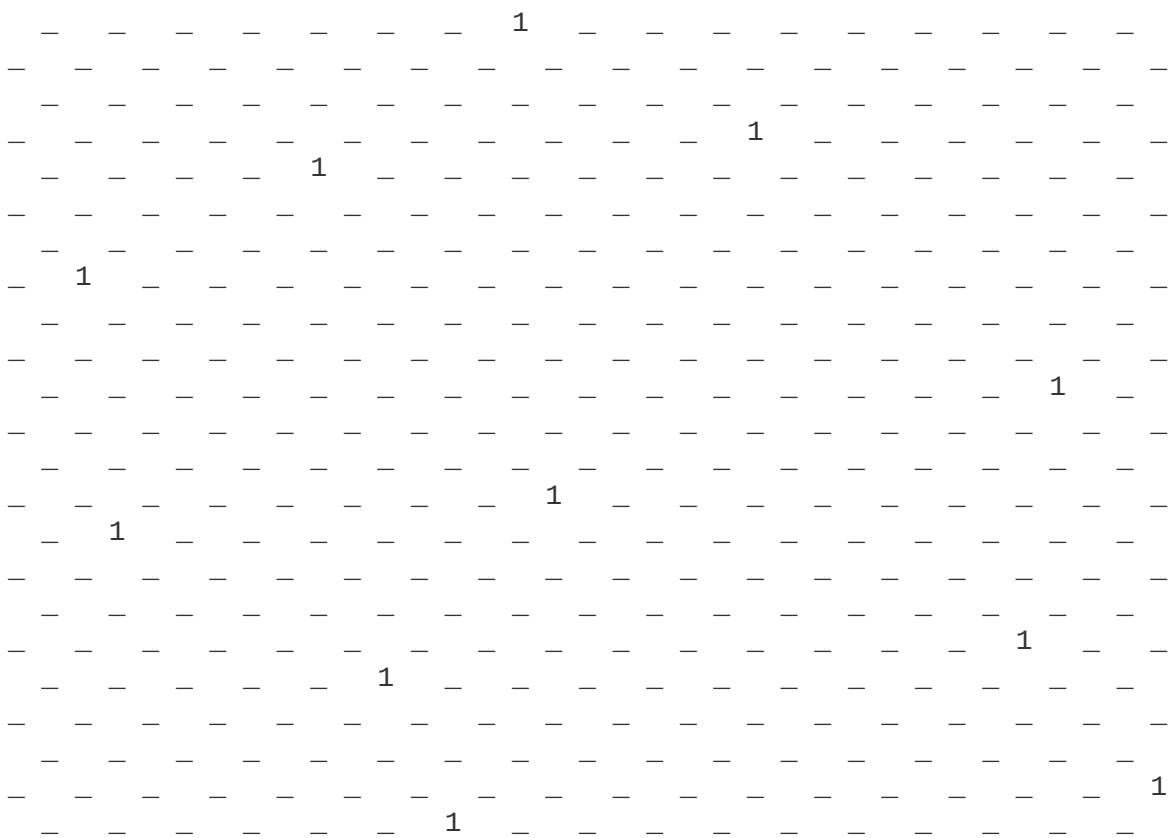
5=26a13be56a3d58249aecbeeb069a3a71fb403e26,
6=45cb1f3dce7ee1f8ee57bee176c98daa422d5d7f,
7=ee3bd7e14300ac355cf1105e4113ea9317b9b5cf,
8=a73df4c1322eab3c5573e406a65874e651ce9587,
9=13c9dd9c91c827c0c4ccd56acd1bdcc7ea5720d1,
10=7b36a4f5fc8fb5a6e379ae9ba5093546804ff2dd,
11=6c7f1cde8647fce432e9c1a6f08d633a5072be15,
12=281e2d2269ff40b3ff5cfe79a4393d9058183e5a,
13=e489daa692c9e1fd9a51c8acb16ac105a840ef68,
14=c82e5e89c9042306f560443c54d34be029ea8715,
15=b4638cf378282b954a7b83173d499361553d3cad,
16=5d3c161b08bbe5fbba94dcc6af80e032c7a4441e,
17=5a4b5f780c4373cedbcf930913ad654c3b694c3c,
18=506660783af019add4343d633af52c25929fc710,
19=1dba846ab0bc772d18d19f761b9835d98895ea9b,
20=5d82f9dea92eb8844e5645c36c0f31ab6667aaeb,
21=e6601b76064582b8731174ee230056bdf66d51f2,
22=d0ef7a356f1e36badb9f6730edf77e1b6edbf496
}
// Data matrix (matching coefficients between source and target table
columns)
//=====
=====
[
[71,59,0,11,90,91,85,90,0,0,66,0,0,0,0,14,29,0,0,0,0,0,0,0,0,19,14,42,
17,19,17,54,0,0],
[43,99,0,27,100,100,100,100,0,0,99,0,0,0,0,45,80,19,0,0,0,0,0,0,0,0,48
,99,30,0,52,96,0,0],
[62,33,0,0,79,81,77,79,0,0,40,0,0,0,0,0,17,0,0,0,0,0,0,0,0,11,25,0,21,23
,23,0,32,0,0],
[88,100,0,32,100,100,100,100,0,0,100,32,0,0,0,68,92,0,12,0,36,48,0,0,0,0
,96,92,100,96,68,96,100,28,68],
[0,100,0,27,100,100,100,100,72,0,100,0,0,0,0,54,100,0,0,0,0,72,0,0,0,18,
36,100,100,100,100,81,100,0,100],
[46,31,0,22,66,0,22,0,0,0,57,24,0,0,0,40,35,0,0,0,0,0,0,60,0,46,37,0,0,24,
31,84,0,0,46,100],
[45,29,0,25,65,0,20,0,0,0,56,22,0,0,0,38,34,0,0,0,0,0,59,0,45,36,0,0,22,
29,84,0,0,45,100],
[86,100,0,14,99,100,98,99,0,0,100,0,0,0,0,20,34,0,0,0,0,0,0,0,0,0,0,0,57
,0,15,21,98,0,0],
[62,64,0,16,93,89,91,90,0,0,72,0,0,0,0,23,45,0,0,0,0,0,0,0,35,0,0,24,55,
18,16,24,69,0,0],
[70,64,0,15,92,96,86,93,0,0,72,0,0,0,0,20,34,0,0,0,0,0,0,0,0,0,0,13,41,1
1,21,20,63,0,0],
[58,56,0,32,91,82,90,89,0,0,63,0,0,0,0,27,36,18,0,0,0,0,46,0,20,0,27,23,
45,64,21,28,54,0,79],
[62,33,0,0,79,81,77,78,0,0,40,0,0,0,0,17,0,0,0,0,0,0,0,0,11,25,0,21,23
,23,0,32,0,0],
[59,0,0,0,57,61,48,54,0,0,0,14,0,0,0,0,11,0,0,0,0,16,0,0,0,42,91,0,0,38,
33,0,0,0,0],

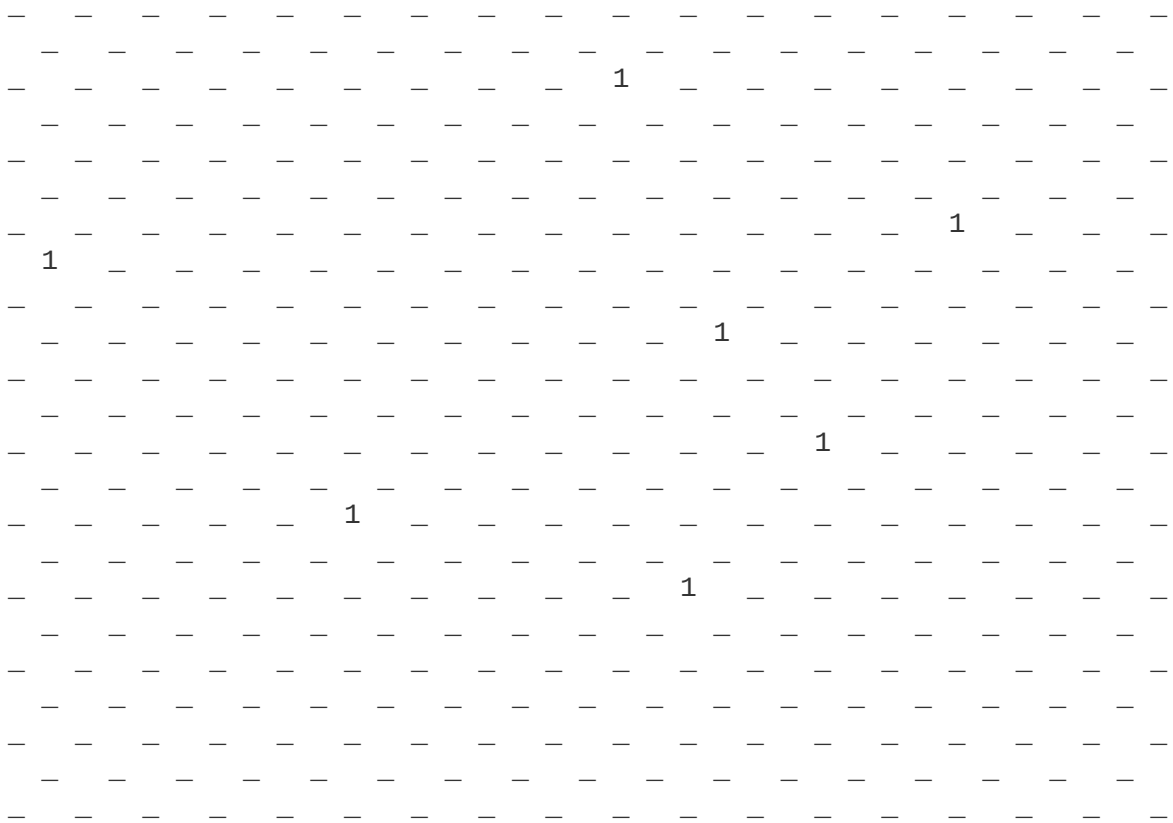
```

// Solution 3

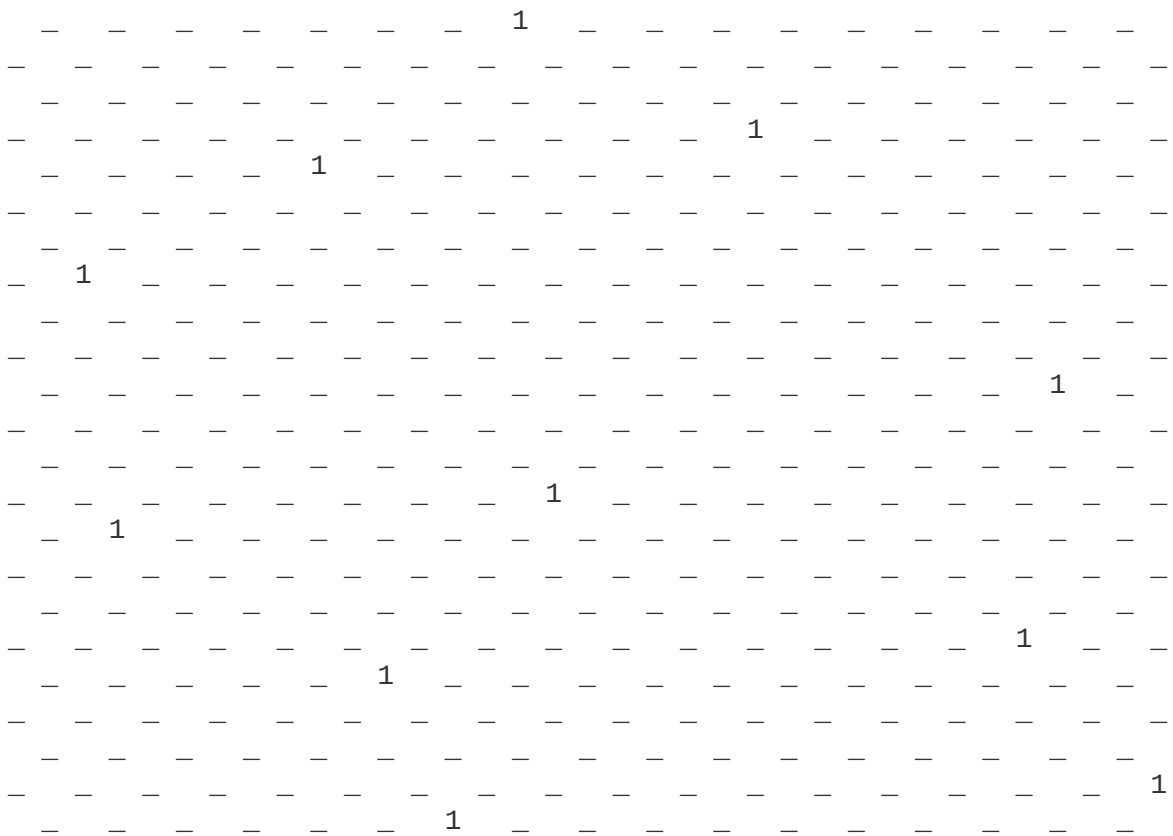
//=====





// Solution 4

//=====




```

import com.redislabs.redisgraph.ResultSet;
import com.google.ortools.Loader;
import com.google.ortools.sat.CpModel;
import com.google.ortools.sat.CpSolver;
import com.google.ortools.sat.CpSolverSolutionCallback;
import com.google.ortools.sat.IntVar;
import com.google.ortools.sat.LinearExpr;
import com.iotahoe.processors.api.cp.Column;
import com.iotahoe.processors.api.cp.CpMdsGraphClient;
import com.iotahoe.processors.api.cp.CpUtils;
import com.iotahoe.processors.api.cp.CypherQuery;
import com.iotahoe.util.AppUtils;
import com.iotahoe.util.ProcessRunnable;
import com.iotahoe.util.TimeUtils;
import java.util.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class CPSMapping implements ProcessRunnable {
    private static final Logger logger = LoggerFactory.getLogger
(CPSMapping.class);
    String lettuce_host      = "redis://localhost";
    String redis_host        = "localhost";
    int redis_port           = 6379;
    String rdb_column_query = "";
    private List<long[][]> matrix_list      = new ArrayList<>();
    private List<Map<String, String>> map_list = new ArrayList<>();

    private String org_table      = "GRP02_SOURCE0";
    private String big_table      = "BIG_TABLE";
    private String graph_name     = "MDS_GRAPH";
    private String pivot_matrix   = "pivot_matrix";
    private String pivot_map      = "pivot_map";
    private String mask_hash      = "mask_index";
    private int row_ub = 2;
    private int col_ub = 2;
    private int threshold = 10;
    private boolean match = true;
    private Map<String, Integer> mask_map = new HashMap<>();

    public static void main(String[] args) throws Exception {
        boolean debug = true;
        if (debug) {
            debugMain(args);
        } else {
            new CPSMapping().run(ProcessUtils.parseCommandArgs(args
[0]));
        }
    }
    @Override

```

```

        public void run(Map<String, Object> params) throws
NumberFormatException {
            Loader.loadNativeLibraries();
            long startNanos = TimeUtils.start();
            if (params != null) {
                logger.info("run: params={} \n", params);
                this.redis_host      = AppUtils.getStringOrThrow(params,
"redis_host");
                this.lettuce_host    = AppUtils.getStringOrThrow(params,
"lettuce_host");
                this.org_table       = AppUtils.getStringOrThrow(params,
"org_table");
                this.big_table       = AppUtils.getStringOrThrow(params,
"big_table");
                this.graph_name      = AppUtils.getStringOrThrow(params,
"graph_name");
                this.pivot_map       = AppUtils.getStringOrThrow(params,
"pivot_map");
                this.pivot_matrix    = AppUtils.getStringOrThrow(params,
"pivot_matrix");
                this.mask_hash       = AppUtils.getStringOrThrow(params,
"mask_hash");
                this.redis_port      = AppUtils.getIntOrNull(params,
"redis_port");
                this.row_ub          = AppUtils.getIntOrNull(params,
"row_ub");
                this.col_ub          = AppUtils.getIntOrNull(params,
"col_ub");
                this.threshold       = AppUtils.getIntOrNull(params,
"threshold");
                this.match           = AppUtils.getOrThrow(params, "match");
            }
            // Queries to the Graph

//=====
            String bt_query = String.format(CypherQuery.BT_COL_SET,
org_table, big_table);
            String ot_query = String.format(CypherQuery.OT_COL_SET,
org_table, 0);
            logger.info(bt_query + "\n");
            logger.info(ot_query + "\n");
            Map<String, Column> bt_col_set = new HashMap<>();
            Map<Integer, String> bt_col_idx = new HashMap<>();
            Map<String, Column> ot_col_set = new HashMap<>();
            Map<Integer, String> ot_col_idx = new HashMap<>();

            // Building constraints and model

//=====
            CpModel model = new CpModel();

```



```

        IntVar[][] coeff;
        try (CpMdsGraphClient mds_client = new CpMdsGraphClient
(redis_host, redis_port)) {
            ResultSet rs_bt = mds_client.runCypherQuery(this.
graph_name, bt_query);
            // Create big_table column mapping and inverted index

//=====
            int _bt_idx = 0;
            while (rs_bt.hasNext()) {
                Column column = new Column(CpUtils.convert(rs_bt.
next()));
                String id = column.getId();
                if (bt_col_set.get(id) == null) {
                    column.setOrdNum(_bt_idx);
                    bt_col_set.put(id, column);
                    bt_col_idx.put(_bt_idx, id);
                    _bt_idx++;
                }
            }
            final int bt_idx = _bt_idx;
            logger.info(new Gson().toJson(bt_col_idx) + "\n");

            ResultSet rs_ot = mds_client.runCypherQuery(graph_name,
ot_query);
            // Create original_tables column mapping and inverted index
            for each original table

//=====
            int _ot_idx = 0;
            while (rs_ot.hasNext()) {
                Column column = new Column(CpUtils.convert(rs_ot.
next()));
                String id = column.getId();
                if (ot_col_set.get(id) == null) {
                    column.setOrdNum(_ot_idx);
                    ot_col_set.put(id, column);
                    ot_col_idx.put(_ot_idx, id);
                    _ot_idx++;
                }
            }
            final int ot_idx = _ot_idx;
            logger.info(new Gson().toJson(ot_col_idx) + "\n");

            // Building data matrix

//=====
            long[][] data = new long[ot_col_idx.size()][bt_col_idx.
size()];

```

```

        ot_col_idx.keySet().forEach((Integer i) -> {
            String i_hll = ot_col_set.get(ot_col_idx.get(i)).
getHllId();
            Arrays.fill(data[i], 0);
            bt_col_idx.keySet().forEach(j -> {
                String j_hll = bt_col_set.get(bt_col_idx.get(j)).
getHllId();
                // Boolean flag switches between hll cardinality
and match coefficient
                // - true, is using match;
                // - false, is using hll cardinality.
                data[i][j] = mds_client.cardHllIntersection(i_hll,
j_hll, threshold, match);
            });
        });
        logger.info(new Gson().toJson(data));

        // Mask structurally the same as a pivot hash, but value is
in {0, 1}
        // Mask hash name has prefix "mask": "mask:GRP02_SOURCE2:3:
3:10";
        // and pivot map: "pivot:GRP02_SOURCE2:3:3:10"

//=====
        this.mask_map = mds_client.getMaskMap(ot_col_set,
bt_col_set, hashId(mask_hash));

        logger.info("MASK_MAP: " + new Gson().toJson(mask_map));

        // Build coeff matrix and sum vector for matrix rows and
columns.

//=====
        coeff = new IntVar[ot_idx][bt_idx];
        for (int i = 0; i < ot_idx; i++) {
            for (int j = 0; j < bt_idx; j++) {
                String cell = i + "_" + j;
                if (mask_map.get(cell) == null) {
                    coeff[i][j] = model.newIntVar(0, 1, cell);
                } else {
                    // Setting IntVar as a constant
                    int v = mask_map.get(cell);
                    coeff[i][j] = model.newIntVar(v, v, cell);
                }
            }
        }
        // Sum vector should compensate
        // possible mismatch between ot_column cardinality and
cardinality of
        // bt_column and ot_column intersection.

```

```

//=====
    IntVar[] sum_row = new IntVar[ot_idx];
    for (int j = 0; j < ot_idx; j++) {
        int ub = ot_col_set.get(ot_col_idx.get(j)).getCard();
        sum_row[j] = model.newIntVar(0, ub * row_ub, "sum_row_"
+ j));
    }
    IntVar[] sum_col = new IntVar[bt_idx];
    for (int j = 0; j < bt_idx; j++) {
        int ub = bt_col_set.get(bt_col_idx.get(j)).getCard();
        sum_col[j] = model.newIntVar(0, ub * col_ub, "sum_col_"
+ j));
    }
    logger.info("coeff_by_row count: " + coeff.length);
    logger.info("coeff_by_col count: " + coeff[0].length);

    // Building constraints

//=====
    // 1. Sum of coefficints in each row should be equal 1 (ot
column can be
    // mapped only to one bt column).
    // In fuzzy mapping this number can be greater than 1;
    //
    // 2. Scalar product of data row on oo intersections of bt
and ot columns
    // should be equal ot column cardinality minus delta.
    // Cardinality of intersection cannot be greater than
cardinality of any set
    // in intersection.

//=====
    for (int i = 0; i < ot_idx; i++) {
        // 0. Create IntVar array by row
        IntVar[] bt_idx_vars = new IntVar[bt_idx];
        System.arraycopy(coeff[i], 0, bt_idx_vars, 0, bt_idx);
        // 1.
        model.addLessOrEqual(LinearExpr.sum(bt_idx_vars),
col_ub);
        model.addGreaterOrEqual(LinearExpr.sum(bt_idx_vars), 0);
        // 2.
        model.maximize(LinearExpr.scalProd(bt_idx_vars, data
[i]));
        model.addEquality(// scalarProduct should be equal to
delta
            LinearExpr.scalProd(bt_idx_vars, data[i]),
            // delta scope is defined as [0,
card_of_ot_column].
            sum_row[i]

```

```

        );
    }
    long[][] t_data = CpUtils.transposeMatrix(data);
    for (int i = 0; i < bt_idx; i++) {
        // 0. Create IntVar array by col
        IntVar[] ot_idx_vars = new IntVar[ot_idx];
        for (int j = 0; j < ot_idx; j++) {
            ot_idx_vars[j] = coeff[j][i];
        }
        // 1.
        model.addLessOrEqual(LinearExpr.sum(ot_idx_vars),
row_ub);

        model.addGreaterOrEqual(LinearExpr.sum(ot_idx_vars), 0);
        // 2.
        model.maximize(LinearExpr.scalProd(ot_idx_vars, t_data
[i]));

        model.addEquality(// scalarProduct should be equal to
delta

            LinearExpr.scalProd(ot_idx_vars, t_data[i]),
            // delta scope is defined as [0,
card_of_ot_column].

            sum_col[i]

        );
    }
    // Create a solver and solve the model.
    CpSolver solver = new CpSolver();
    VarArraySolutionPrinter cb = new VarArraySolutionPrinter
(data, coeff, bt_col_idx, ot_col_idx);
    solver.solveWithSolutionCallback(model, cb);
    logger.info("Statistics");
    logger.info(" - conflicts : " + solver.numConflicts());
    logger.info(" - branches : " + solver.numBranches());
    logger.info(" - wall time : " + solver.wallTime() + " s");
    logger.info(" - obj value : " + cb.objectiveValue());
    logger.info(" - solutions : " + cb.solutionCount);
    this.setMatrix_list(cb.getMatricesList());
    this.setMap_list(cb.getIdMap());
    // As we mentioned, mask and pivot hashes are structurally
similar

    // and differ only by prefix
    mds_client.mergePivotMap(cb.getIdMap(), hashId(this.
pivot_map));
    } catch (Exception e) {
        logger.error(e.getMessage());
    }
    logger.info("\n\nrunOnce: finished; seconds={}\n\n", TimeUtils.
stopSeconds(startNanos));
}
private String hashId(String prefix) {
    return prefix + ":" + org_table + ":" + row_ub + ":" + col_ub +

```

```

":" + threshold;
    }
    public static void debugMain(String[] args) {
        long startNanos = TimeUtils.start();
        Loader.loadNativeLibraries();

        // mask_map keep list of coeff matrix cells that hold
constants, so
        // range of values is [0, 0] or [1, 1].

//=====
        boolean matrix_out = true;
        CPSMapping runner = new CPSMapping();
        runner.run(null);
        if (matrix_out) {
            CpUtils.printListOfMatrices(runner.getMatrix_list());
        } else {
            System.out.println(new Gson().toJson(runner.
getMapId_list()));
        }
        logger.info("\n\nrunOnce: finished; seconds={} \n\n", TimeUtils.
stopSeconds(startNanos));
        System.exit(0);
    }
    static class VarArraySolutionPrinter extends
CpSolverSolutionCallback {
        private int solutionCount;
        private final long[][] data;
        private final IntVar[][] vars;
        private final int n_cols;
        private final int n_rows;
        private final Map<Integer, String> bt_col_idx;
        private final Map<Integer, String> ot_col_idx;
        private final List<Map<String, String>> list_id_map = new
ArrayList<>();
        private final List<long[][]> list_matrices;
        public VarArraySolutionPrinter(long[][] data, IntVar[][]
variables, Map<Integer, String> bt_col_idx, Map<Integer, String>
ot_col_idx) {
            vars = variables;
            n_cols = variables[0].length;
            n_rows = variables.length;
            list_matrices = new ArrayList<>();
            this.bt_col_idx = bt_col_idx;
            this.ot_col_idx = ot_col_idx;
            this.data = data;
        }
        @Override
        public void onSolutionCallback() {
            long[][] map = new long[n_rows][n_cols];

```

```

        Map<String, String> id_map = new HashMap<>();
        for (int i = 0; i < n_rows; i++) {
            for (int j = 0; j < n_cols; j++) {
                int v = (int) value(vars[i][j]);
                if (v > 0) {
                    map[i][j] = data[i][j];
                    // Build source / target matrix
                    String source = ot_col_idx.get(i);
                    String target = bt_col_idx.get(j);
                    id_map.put(source + ":" + target, String.valueOf
(data[i][j]));
                }
            }
        }
        list_matrices.add(map);
        list_id_map.add(id_map);
        solutionCount++;
    }
    public List<long[][]> getMatricesList() {
        return list_matrices;
    }
    public List<Map<String, String>> getIdMap() {
        return list_id_map;
    }
    public int getSolutionCount() {
        return solutionCount;
    }
}
public List<long[][]> getMatrix_list() {
    return matrix_list;
}
public void setMatrix_list(List<long[][]> matrix_list) {
    this.matrix_list = matrix_list;
}
public List<Map<String, String>> getMapId_list() {
    return map_list;
}
public void setMap_list(List<Map<String, String>> map_list) {
    this.map_list = map_list;
}
}

```

1. Data as matrix:

ort - original table column;
dlt - data lake table column.

ort \ dlt	b ₁	b ₂		b _j		
a ₁	(1,1)	(1,2)				
a ₂	(2,1)	(2,2)				
a _i				(i,j)		

$a_i \in A$, A is ort dataset.

$b_j \in B$, B is dlt dataset.

(a_i, b_j) is a cardinality

$\text{card}(A \cap B)$

With HLL:

$\text{card}(A) + \text{card}(B) -$

$\text{card}(A \cup B) =$

$\hookrightarrow \text{merge}(\text{HLL}(A), \text{HLL}(B))$

2. Variables:

Matrix $(m, n) \rightarrow$

	b ₁	b ₂		b _j		
a ₁	x ₁₁	x ₁₂		x _{1j}		
a ₂	x ₂₁	x ₂₂		x _{2j}		
a _i	x _{i1}	x _{i2}		x _{ij}		

m = number of dlt columns that have $\text{card}(a_i \cap b_j) > 0$; $b_j \in B$
 n = number of columns in ort table.

$x_{ij} = \{0, 1\}$ has only two values 0 and 1

3. Constraints:

3.1. Row constraints

$$(*) \text{card}(a_i) = a_i * x_{i1} + x_{i2} * a_i * x_{ij} + \dots + a_i * x_{im} + \epsilon_i$$

$\sum_{j=1}^m x_{ij} = 1$; ϵ is a compensation value, to make
It means that in the equation (*) we can have only one non 0 x_{ij}